

Ingeniería en Computación

ELECTRÓNICA DIGITAL

Teórico 4

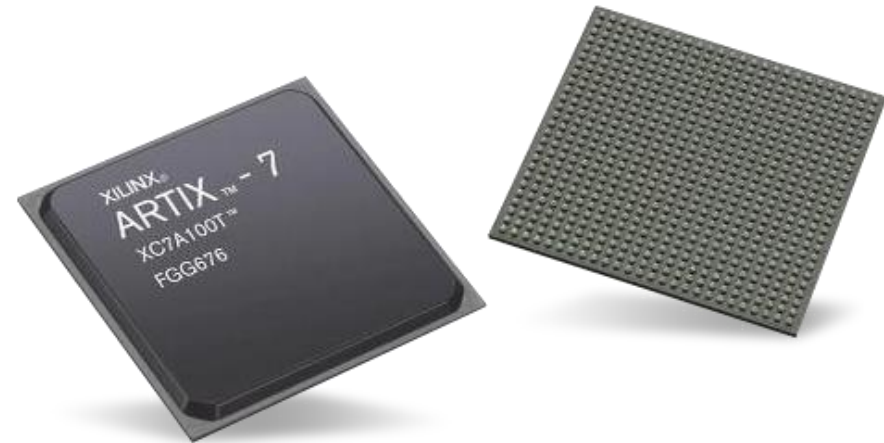
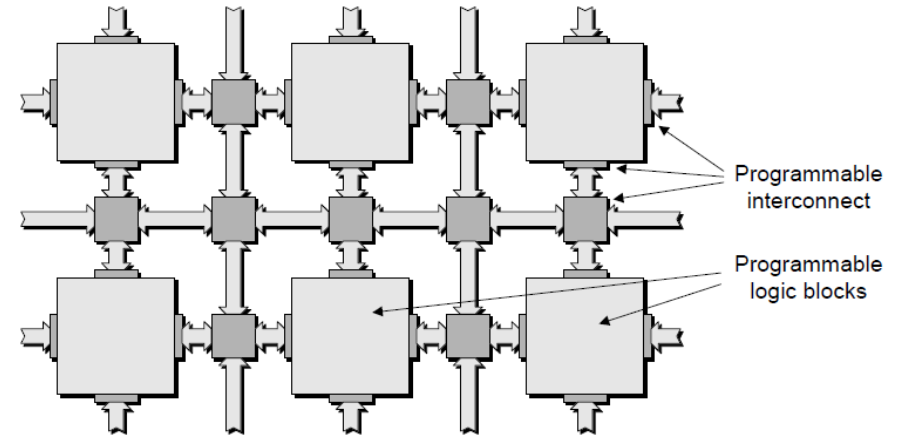
Introducción a FPGA y VHDL



Universidad Nacional
de **Río Negro**

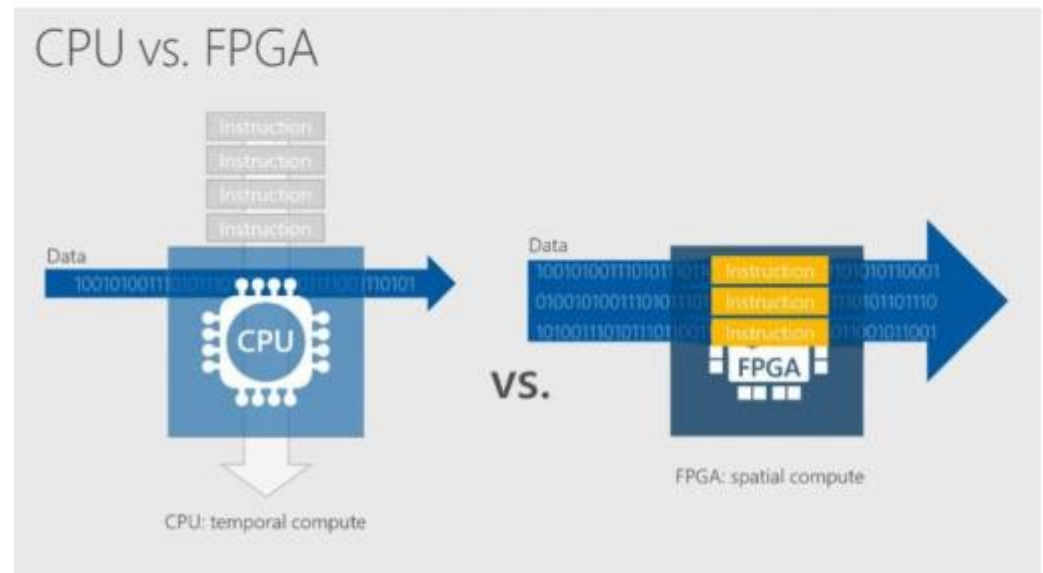
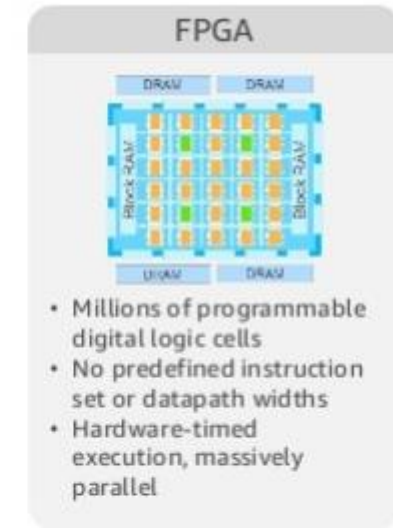
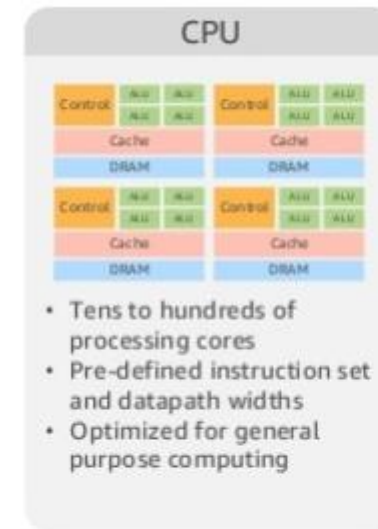
¿Qué es una FPGA?

- Circuitos Integrados que contienen bloques configurables de lógica junto con conexiones configurables entre esos bloques.
- Las FPGA se programan *“in the field”*, o sea, no las programa el fabricante, sino que la puede programar el desarrollador *“en el campo”*.
- **FPGA:** *“Field-programmable gate array”* (matriz de compuertas lógicas programable en campo)



Diferencias con CPU

- CPU: esta diseñado para tareas de propósito general, atado a un set de instrucciones específico.
- FPGA: pensado para diseños específicos que buscan el mayor rendimiento posible.
- El CPU funciona secuencialmente, mientras que la FPGA permite paralelizar el procesamiento.



Características de las FPGA

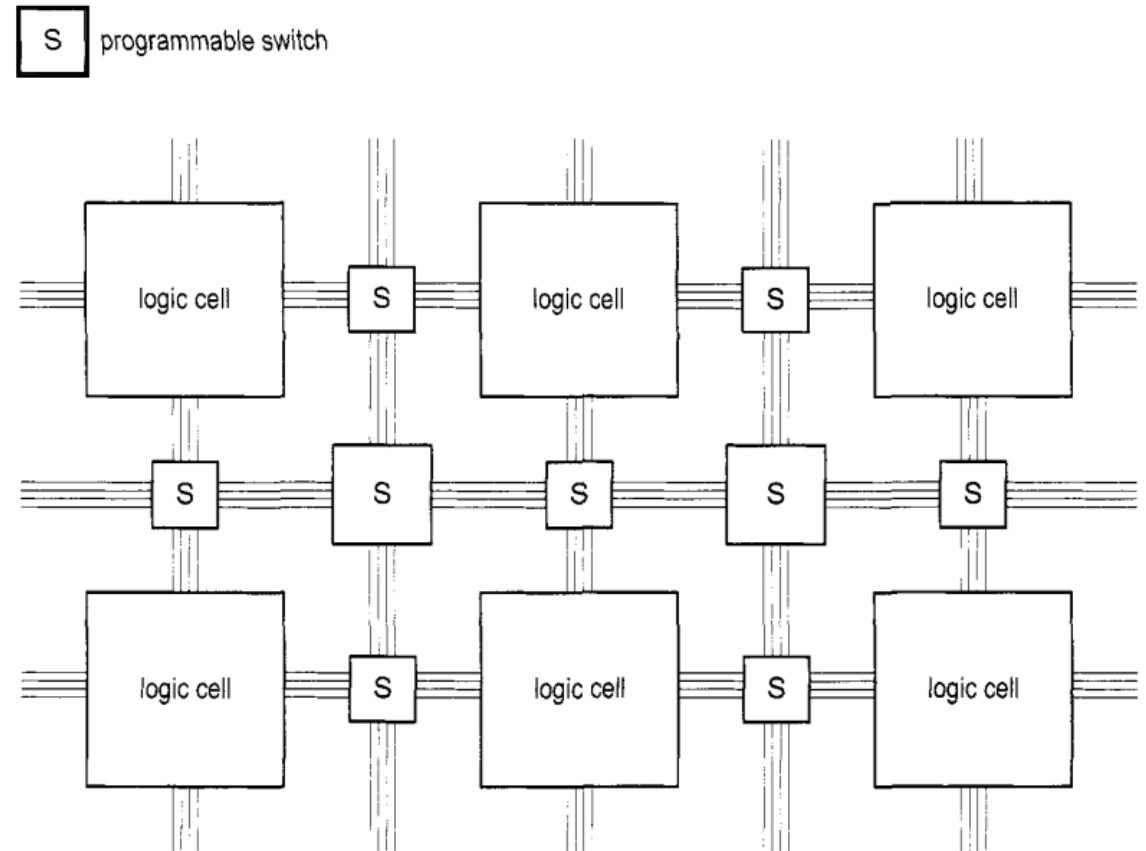
- Alto grado de paralelismo y rendimiento en procesamiento de señales
- Asignación de puertos flexibles
- Interfaces multi-estándar
- Reconfigurabilidad y reutilización
- Escalabilidad en el flujo de desarrollo
- Menor tiempo de llegada al mercado vs. ASIC (circuito integrado de aplicación específica, *chips*)

¿Para qué se usan?

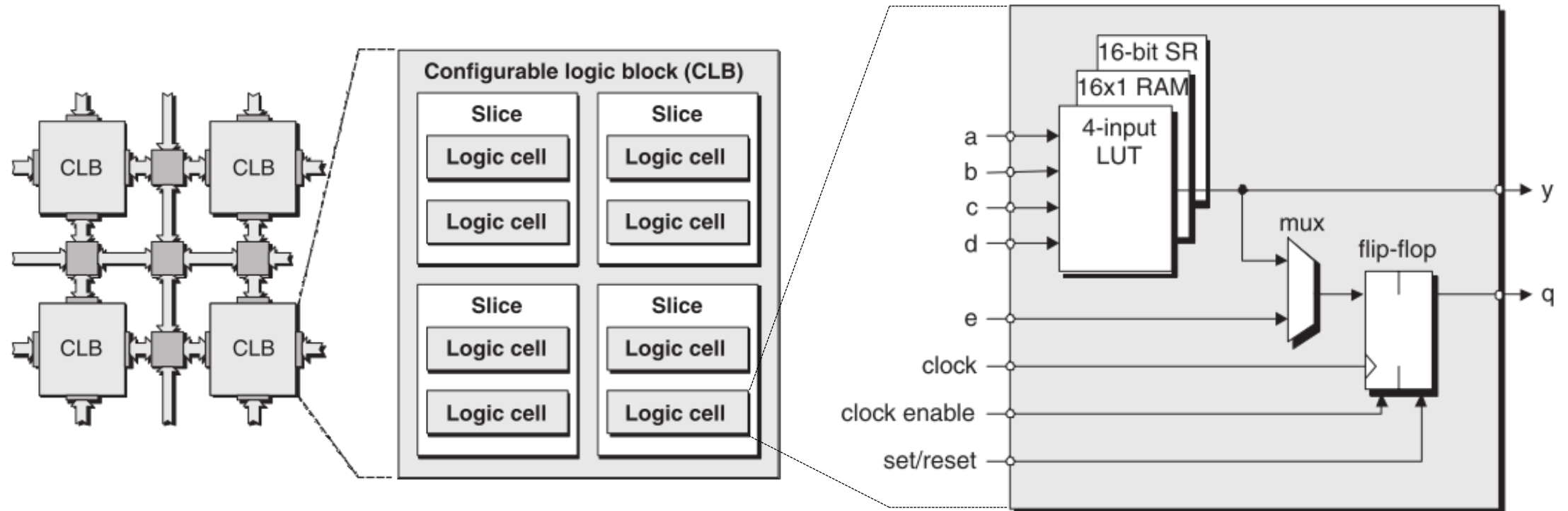
- Prototipado y verificación de ASICs (Circuitos Integrados de Aplicaciones Específicas)
- Aplicaciones de Hardware muy especiales y de mediana escala de producción
 - Industria aeroespacial, radar, nuclear, etc.
- Procesamiento de señales (DSP)
 - Procesamiento en tiempo real
 - Adquisición de datos
- Aplicaciones de todo tipo, drones, autos, electrónica de consumo, criptografía, automatización, etc.

Arquitectura

- Bloques Lógicos
- Matriz de Ruteo y Señales Globales
- Bloques I/O
- Recursos de Reloj
- Memoria Embebida
- Bloques Multiplicadores, Sumadores, DSP
- Características Avanzadas:
 - Procesadores hard embebidos, etc.



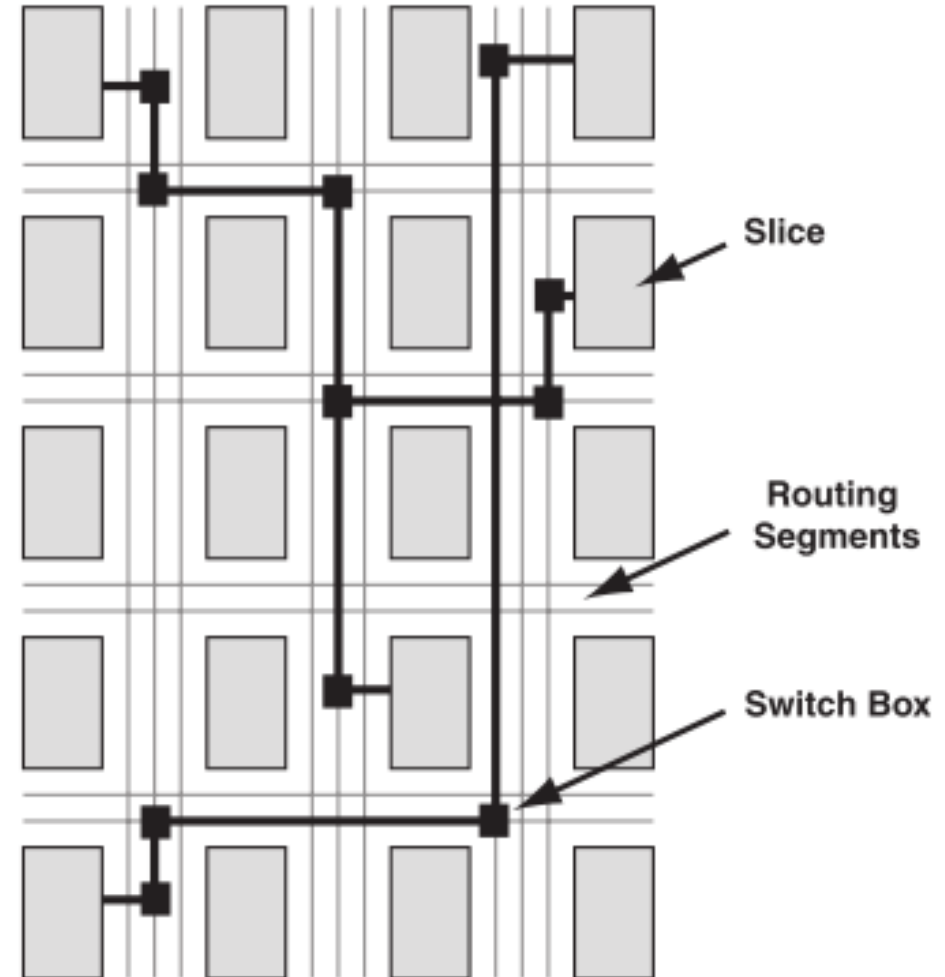
Bloques Lógicos



LUT: LookUp Table

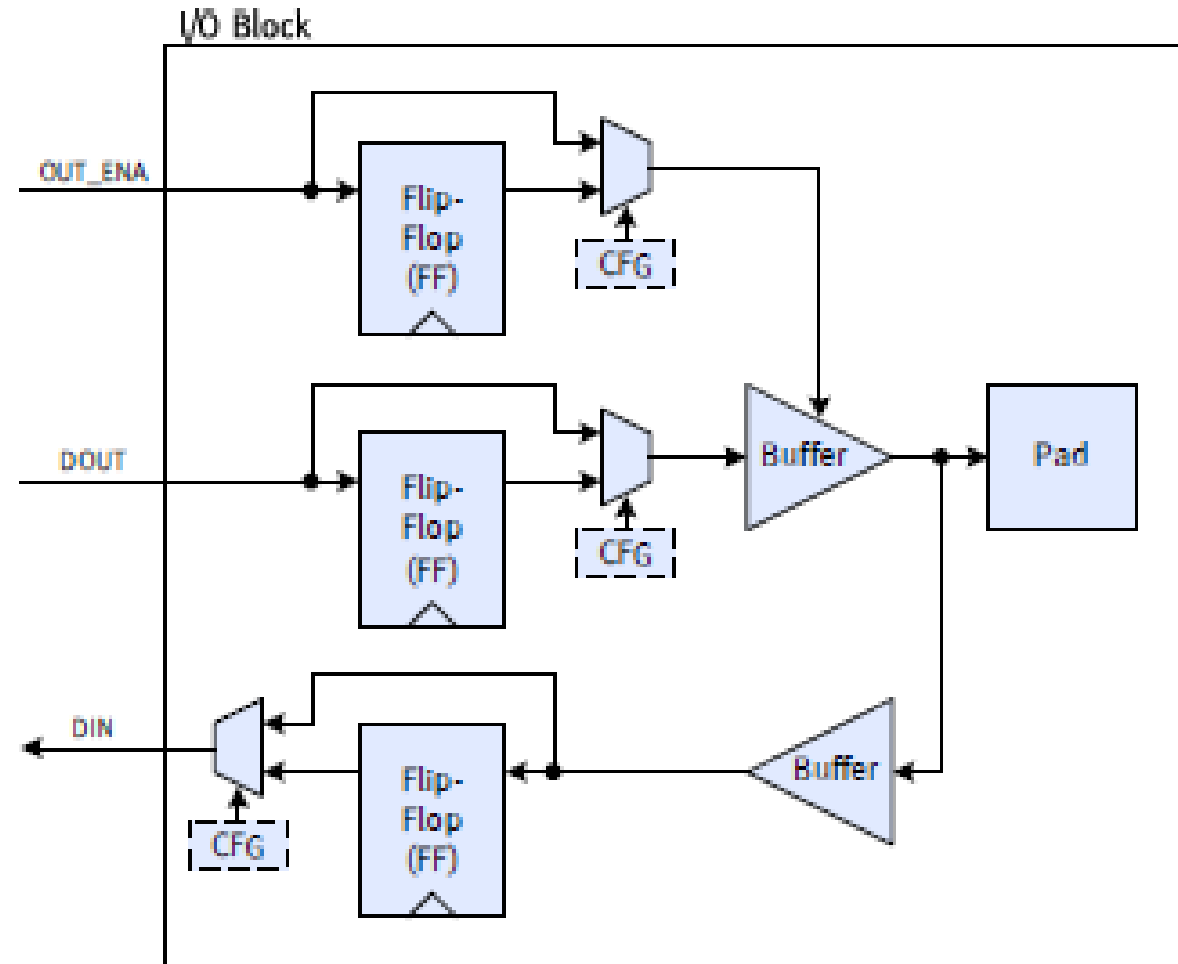
Matriz de Ruteo y Señales Globales

- Canales de Ruteo Horizontales/Verticales y switches programables.
- Global low-skew routing.



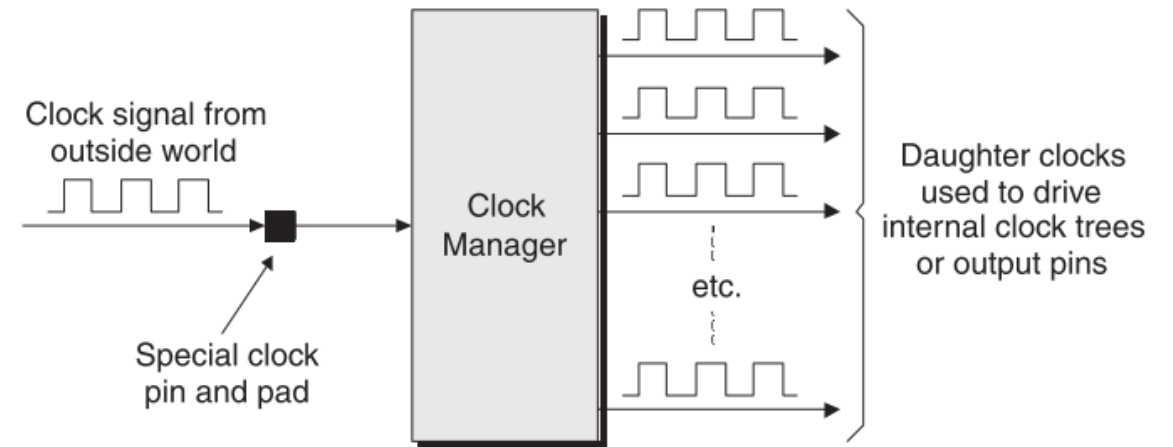
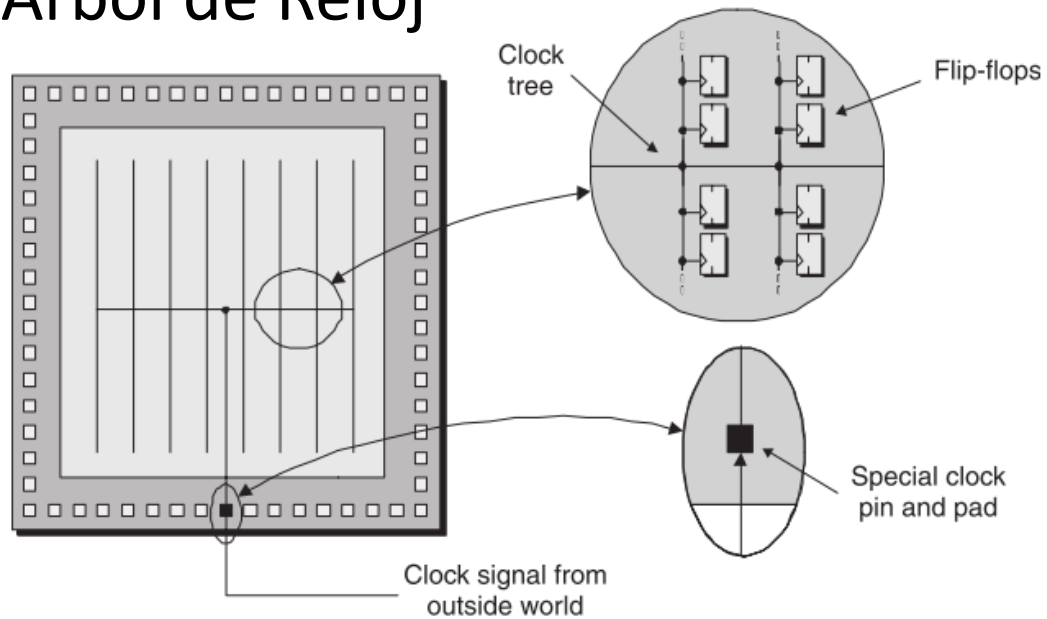
Bloques I/O (Entradas/Salidas)

- Dirección (I,O,bi)
- Data Rate (SDR, DDR, etc)
- Voltaje (1.2 V a 3.3 V)



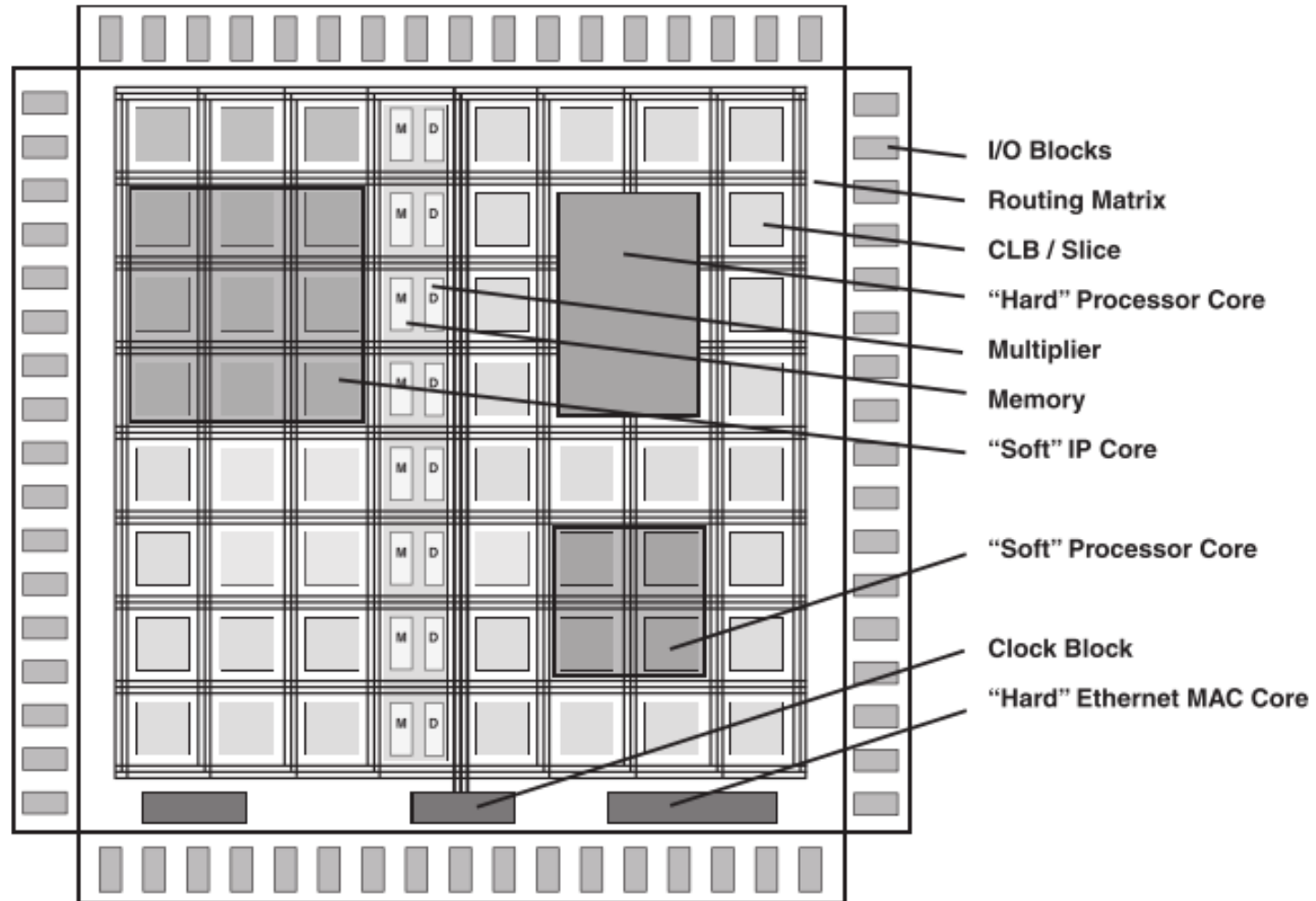
Recursos de Reloj

■ Árbol de Reloj



■ Digital Clock Manager

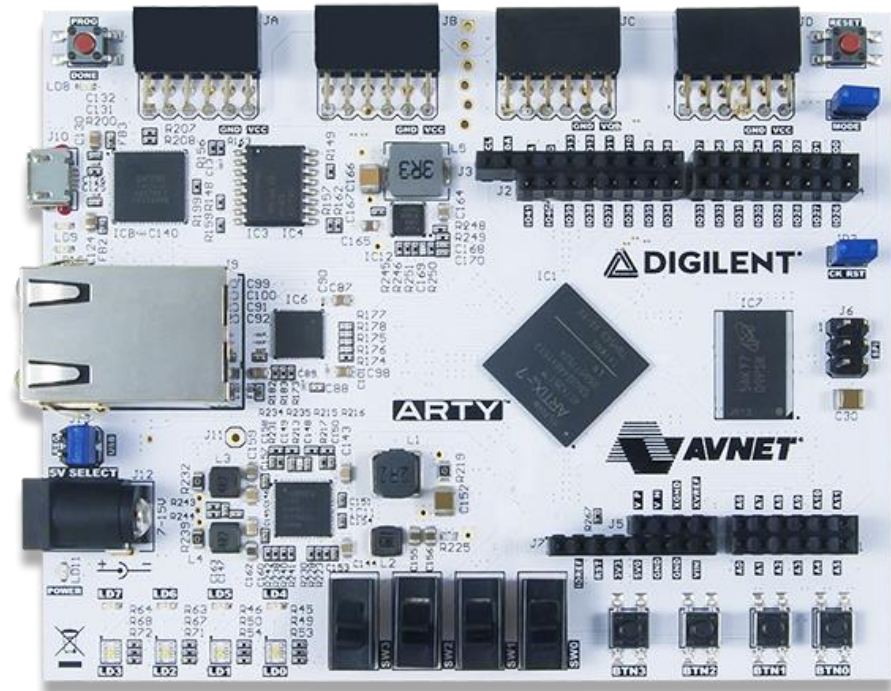
Arquitectura: La Figura Completa



Arquitectura: La placa completa Arty A7

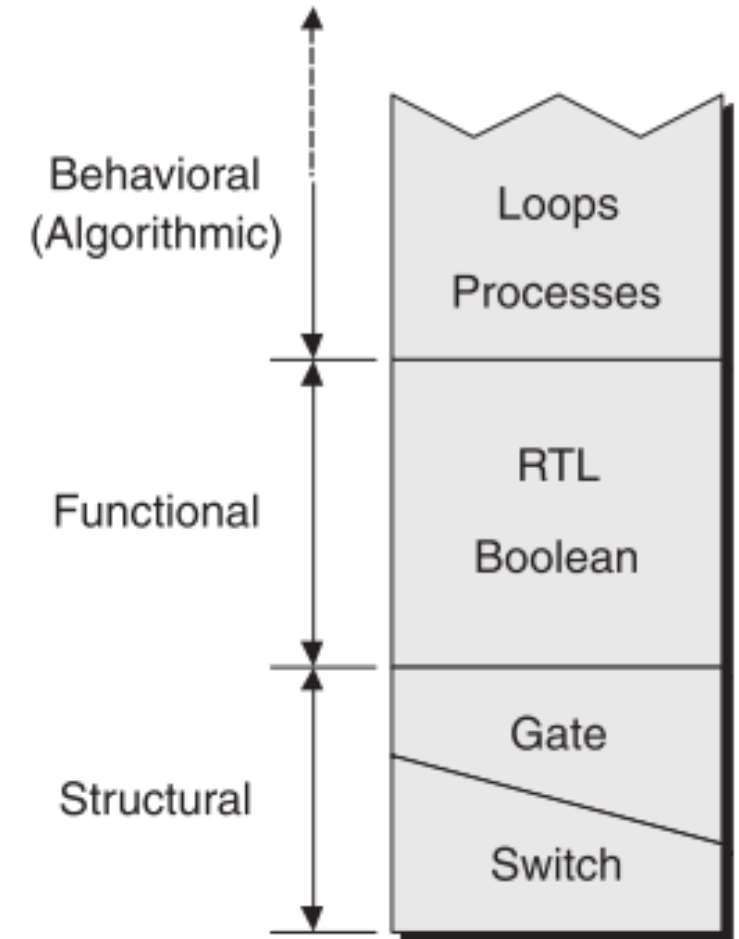
Features

- Xilinx Artix-7 FPGA
 - 5,200 slices containing four 6-input LUTs and 8 flip-flops (15,850 slices)¹⁾
 - 1,800 Kbits of fast block RAM (4,860 Kbits)²⁾
 - 5 clock management tiles (CMTs), each with a phase-locked loop and mixed-mode clock manager (6 CMTs)³⁾
 - 90 DSP slices (240 DSP slices)⁴⁾
 - Internal clock speeds exceeding 450MHz
 - On-chip analog-to-digital converter (XADC)
 - Programmable over JTAG and Quad-SPI Flash
- System Features
 - 256MB DDR3L with a 16-bit bus @ 667MHz
 - 16MB Quad-SPI Flash
 - USB-JTAG Programming circuitry
 - Powered from USB or any 7V-15V source
- System Connectivity
 - 10/100 Mbps Ethernet
 - USB-UART Bridge
- Interaction and Sensory Devices
 - 4 Switches
 - 4 Buttons
 - 1 Reset Button
 - 4 LEDs
 - 4 RGB LEDs
- Expansion Connectors
 - 4 Pmod connectors
 - Arduino/ChipKit Shield connector



HDL: Lenguaje de Descripción de Hardware

- Visualizar, capturar, depurar (*debuggear*), entender y mantener un diseño a nivel de compuertas lógicas se torna imposible pasado cierta complejidad - Fines de 1980.
- **Lenguajes de Descripción de Hardware:** primeros orientados a nivel compuerta, luego simulación.
- Clave: **herramientas** cada vez mas poderosas de síntesis lógica, mapeo, etc.



Niveles de Abstracción

	typical blocks	signal representation	time representation	behavioral description	physical description
transistor	transistor, resistor	voltage	continuous function	differential equation	transistor layout
gate	and, or, xor, flip-flop	logic 0 or 1	propagation delay	Boolean equation	cell layout
RT	adder, mux, register	integer, system state	clock tick	extended FSM	RT level floor plan
processor	processor, memory	abstract data type	event sequence	algorithm in C	IP level floor plan

VHDL: ¡Hola Mundo!

- Ejemplo: Igualdad de 1 bit
- A nivel de compuertas, o sea, expresado utilizando sólo compuertas lógicas básicas NOT/AND/OR/XOR.
- La expresión funcional en modo suma de productos:
 - $eq = \bar{i0} \cdot \bar{i1} + i0 \cdot i1$

input		output
$i0$	$i1$	eq
0	0	1
0	1	0
1	0	0
1	1	1

VHDL: Combinación de dos acrónimos, VHSIC (Very High Speed Integrated Circuit) y HDL (Hardware Description Language)

VHDL: El Primer Código

- **VHDL** no distingue entre mayúsculas y minúsculas
(*case insensitive*).
- **--**: Indica comentarios.

```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9
10 entity eq1bit is
11
12     port (
13         i0,i1: in std_logic;
14         eq1:  out std_logic
15     );
16 end eq1bit;
17
18 architecture Behavioral of eq1bit is
19     signal p0,p1:std_logic;
20
21 begin
22     -- Suma de dos productos
23     eq1 <= p0 or p1;
24
25     -- Terminos producto
26     p0 <= (not i0) and (not i1);
27     p1 <= i0 and i1;
28
29 end Behavioral;
```

VHDL: Library y Package

- Nos permiten agregar tipos, operadores, funciones, etc.
- VHDL es **fuertemente tipado**: cada objeto tiene que tener un tipo.
- Tiene muchos tipos, pero los **sintetizables** no son tantos!

```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9
10 entity eq1bit is
11     port (
12         i0,i1: in std_logic;
13         eq1:  out std_logic
14     );
15 end eq1bit;
16
17 architecture Behavioral of eq1bit is
18     signal p0,p1:std_logic;
19
20 begin
21     -- Suma de dos productos
22     eq1 <= p0 or p1;
23
24     -- Terminos producto
25     p0 <= (not i0) and (not i1);
26     p1 <= i0 and i1;
27
28 end Behavioral;
```

VHDL: Entity

- Declaración de las **señales I/O** del circuito: **in**, **out**, **inout**.
- **std_logic**:
 - Sintetizables '0', '1', 'Z'.
 - Simulación 'U', 'X'.

```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9
10 entity eq1bit is
11
12     port (
13         i0,i1: in std_logic;
14         eq1:  out std_logic
15     );
16 end eq1bit;
17
18 architecture Behavioral of eq1bit is
19     signal p0,p1:std_logic;
20
21 begin
22     -- Suma de dos productos
23     eq1 <= p0 or p1;
24
25     -- Terminos producto
26     p0 <= (not i0) and (not i1);
27     p1 <= i0 and i1;
28
29 end Behavioral;
```

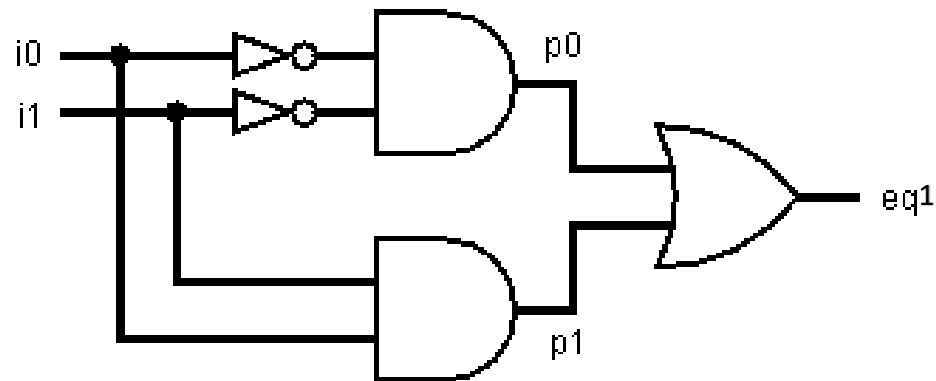
VHDL: Architecture

- Define la **operación del circuito**. Puedo tener más de una arquitectura por entidad.
- Sección de declaración (**signal, constant, etc**)
- Descripción principal: tres **sentencias concurrentes**

$$eq = \bar{i_0} \cdot \bar{i_1} + i_0 \cdot i_1$$

```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9
10 entity eq1bit is
11
12     port (
13         i0,i1: in std_logic;
14         eq1:  out std_logic
15     );
16 end eq1bit;
17
18 architecture Behavioral of eq1bit is
19     signal p0,p1:std_logic;
20
21 begin
22     -- Suma de dos productos
23     eq1 <= p0 or p1;
24
25     -- Terminos producto
26     p0 <= (not i0) and (not i1);
27     p1 <= i0 and i1;
28
29 end Behavioral;
```

VHDL: Representación Gráfica



```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9
10 entity eq1bit is
11
12     port (
13         i0,i1: in std_logic;
14         eq1:  out std_logic
15     );
16 end eq1bit;
17
18 architecture Behavioral of eq1bit is
19     signal p0,p1:std_logic;
20
21 begin
22     -- Suma de dos productos
23     eq1 <= p0 or p1;
24
25     -- Terminos producto
26     p0 <= (not i0) and (not i1);
27     p1 <= i0 and i1;
28
29 end Behavioral;
```

VHDL: ¡Hola Mundo 2!

- Ejemplo 2 : Igualdad de 2 bits
- La expresión funcional en modo suma de productos:

$$\begin{aligned} eq &= \overline{a0} \cdot \overline{a1} \cdot \overline{b0} \cdot \overline{b1} + \\ &+ \overline{a0} \cdot a1 \cdot \overline{b0} \cdot b1 + \\ &+ a0 \cdot \overline{a1} \cdot b0 \cdot \overline{b1} + \\ &+ a0 \cdot a1 \cdot b0 \cdot b1 \end{aligned}$$

input				output
<i>a0</i>	<i>a1</i>	<i>b0</i>	<i>b1</i>	<i>eqab</i>
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

VHDL: El Segundo Código

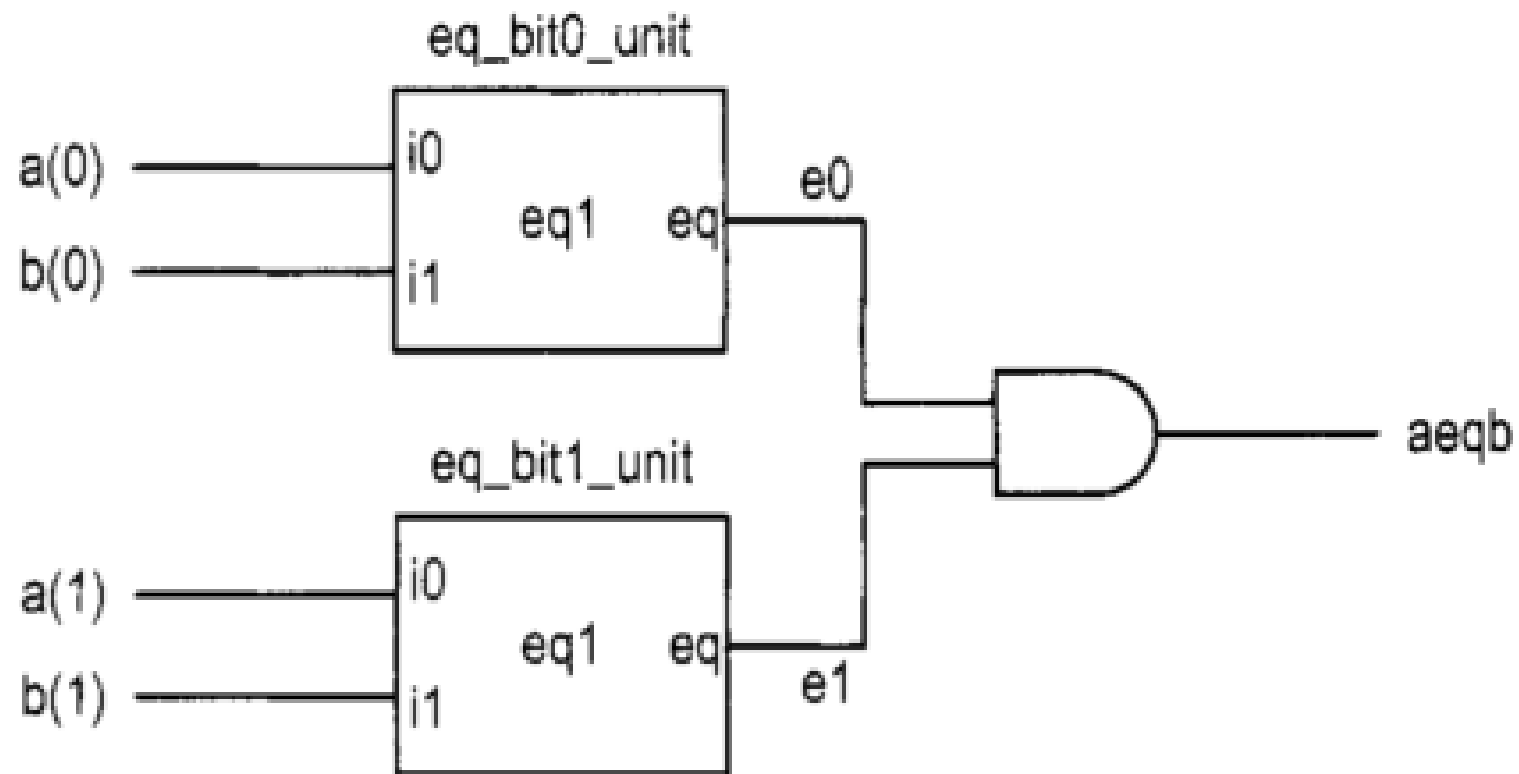
- Lo único nuevo es el tipo de datos **std_logic_vector**.
- Como su nombre lo indica, un vector de **std_logic**.

```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9
10 entity eq2bit is
11     port(
12         a,b: in std_logic_vector(1 downto 0);
13         aeqb: out std_logic
14     );
15 end eq2bit;
16
17 architecture Behavioral of eq2bit is
18     signal p0,p1,p2,p3: std_logic;
19 begin
20     -- Suma de dos productos
21     aeqb <= p0 or p1 or p2 or p3;
22
23     -- Terminos producto
24     p0 <= (not a(0)) and (not a(1)) and
25           (not b(0)) and (not b(1));
26     p1 <= (not a(0)) and a(1) and
27           (not b(0)) and b(1);
28     p2 <= a(0) and (not a(1)) and
29           b(0) and (not b(1));
30     p3 <= a(0) and a(1) and b(0) and b(1);
31 end Behavioral;
```

VHDL: Descripción Estructural (1/2)

- Una misma **entidad** puede tener muchas arquitecturas diferentes.
- Cada **arquitectura** pueden sintetizar a una implementación diferente, o puede ser que dos arquitecturas sean diferentes maneras de expresar la misma implementación.
- Arquitectura estructural:
 - Descripción basada en dividir la entidad en diferentes componentes.
 - **Instanciamos** módulos ya existentes, asignando sus puertos a señales del diseño.

VHDL: Representación Gráfica

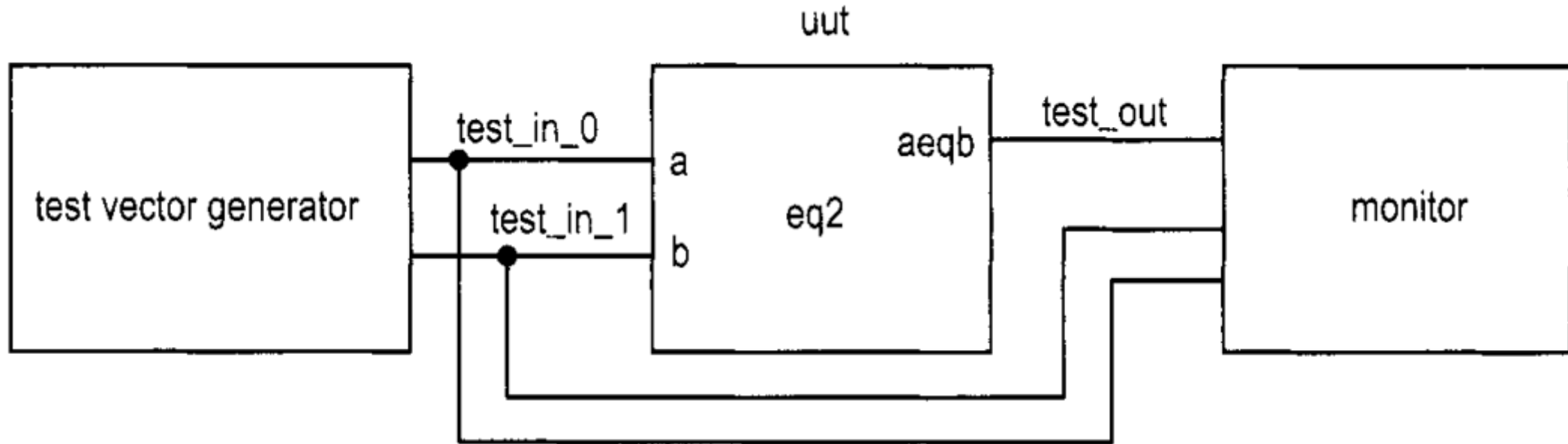


VHDL: Descripción Estructural (2/2)

- El módulo **eq1bit** tiene que estar definido en el proyecto.
- **work**: librería donde están definidos todos los módulos del proyecto.
- Entre paréntesis va la arquitectura que queremos usar *(opcional si el componente tiene una sólo arquitectura)*

```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9
10 entity eq2bit is
11     port(
12         a,b: in std_logic_vector(1 downto 0);
13         aeqb: out std_logic
14     );
15 end eq2bit;
16
17 architecture struct_arch of eq2bit is
18     signal e0, e1 : std_logic;
19 begin
20     -- instanciamos dos comparadores de 1 bit
21     eq_bit0_unit: entity work.eq1bit(Behavioral)
22         port map(i0 => a(0), i1 => b(0), eq1 => e0);
23
24     eq_bit1_unit: entity work.eq1bit(Behavioral)
25         port map(i0 => a(1), i1 => b(1), eq1 => e1);
26
27     -- a y b son iguales si sus bits individuales
28     -- son iguales
29     aeqb <= e0 and e1;
30
31 end struct_arch;
```

VHDL: Testbench (Banco de Pruebas) y Simulación



UUT: Unidad Bajo Prueba

VHDL: ¡Hola Testbench! (1/2)

- **Simulación de Comportamiento** (no de tiempos).
- Inclusión de librerías y definición de entidad igual que antes.
- **Arquitectura:** instanciación del componente, y definición de señales para estimular el diseño y chequear los resultados.

```
1  -----
2  --  Electrónica Digital      --
3  --  Ejemplo 2              --
4  --  Comparador de 2 Bits - Compuertas  --
5  -----
6
7  library ieee;
8  use ieee.std_logic_1164.all;
9
10 ENTITY eq2bit_test IS
11 END eq2bit_test;
12
13 architecture Behavioral of eq2bit_test is
14
15     -- Declaro componente bajo prueba
16     component eq2bit
17     port(
18         a,b : in  std_logic_vector(1 downto 0);
19         aeqb : out std_logic
20     );
21     end component;
22
23     -- Entradas
24     signal test_in_0 : std_logic_vector(1 downto 0) := "00";
25     signal test_in_1 : std_logic_vector(1 downto 0) := "00";
26
27     -- Salidas
28     signal test_out : std_logic;
```

VHDL: ¡Hola Testbench! (2/2)

```
30 begin
31
32     -- Instancio unidad bajo prueba ( uut )
33     uut: eq2bit port map (
34         a => test_in_0,
35         b => test_in_1,
36         aeqb => test_out
37     );
38
39     -- Genero vector de pruebas
40     process
41     begin
42         -- caso 0
43         test_in_0 <= "00";
44         test_in_1 <= "00";
45         wait for 1 ns;
46         assert test_out = '1' report "Fallo caso 0" severity failure;
47
48         -- caso 15
49         test_in_0 <= "11";
50         test_in_1 <= "11";
51         wait for 1 ns;
52         assert test_out = '1' report "Fallo caso 15" severity failure;
53
54         -- si llegué hasta acá, esta OK
55         assert false report "Simulacion OK!" severity note;
56         wait;
57     end process;
58 end;
```